

Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Алтайский государственный технический университет им. И. И. Ползунова»

Факультет информационных технологий
Кафедра прикладной математики

Отчет защищен с оценкой _____

Преподаватель _____
(подпись)

«___» _____ 2026 г.

Отчет
По лабораторной работе №5

«Структуры. Функции. Организация меню»

по дисциплине «Программирование – 2 семестр»

Студент группы ПИ-54 Дворников М. С.

Преподаватель доцент, к.т.н. Егорова Е.В.

Барнаул 2026

№1. Разработка программного обеспечения для организации списка данных об авиарейсах

Каждая строка содержит:

- номер рейса;
- тип самолета;
- пункт назначения;
- дни отправления;
- время вылета;
- время прилета;
- цена билета.

Например: 121 Ту-154 Москва 1,3,5 10.00 14.00 5000.20

Код файла functions.cpp для функций ввода:

```
#define _CRT_SECURE_NO_WARNINGS

#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <math.h>

#include "functions.h"

// Сохранение ввода в переменную и первичная обработка
int input(char** buffer, int len, int dynamic) {
    /*
    buffer: указатель на строку, в которую нужно записать ввод
    len: длина ввода (без учёта '\0', т.е. выделяется len + 1 памяти)
    dynamic: нужно ли выделить память на строку (0, если для buffer уже выделена память - 1, если память не
    выделена)

    Возвращает:
    1: пустая строка (сразу же нажат "Enter")
    0: успешное чтение
    -1: ошибка выделения памяти
    */
    if (dynamic) { // Выделение памяти, если нужно
        *buffer = (char*)calloc(len + 1, sizeof(char));
        if (*buffer == NULL) return -1; // Не хватило памяти
    }

    fgets(*buffer, len + 1, stdin);
    // Передаём len + 1, чтобы fgets получал len символов и после ставил '\0'

    if ((*buffer)[0] == '\n') { // Если пользователь сразу закончил ввод
        if (dynamic) free(*buffer);
        return 1;
    }

    // Очищаем вводный поток, если строка не влезла
    if ((*buffer)[strlen(*buffer) - 1] != '\n')
        while (getchar() != '\n');
    else // Удаляем '\n' с конца
        (*buffer)[strlen(*buffer) - 1] = '\0';

    return 0;
}

// Безопасный ввод int
int int_input(int* buffer) {
    /*
    buffer: указатель на int, в который нужно записать ввод

    Возвращает:
    0: успешное чтение
    */
}
```

```

1: выход из ввода по "###"
2: введена пустая строка
3: есть непрочитанный символ
4: значение ввода выходит за границы int
*/
char user_input[33]; // Буфер для ввода пользователя
fgets(user_input, 33, stdin); // Ввод пользователя
// Очищаем вводной поток, если строчка не влезла
if (user_input[strlen(user_input) - 1] != '\n')
    while (getchar() != '\n');
else // Удаляем '\n' с конца
    user_input[strlen(user_input) - 1] = '\0';

int result = 0; // Результат выполнения функции
long num; // Число пользователя
// Проверяем на выход из ввода "###" - результат 1
if (!strcmp(user_input, "###")) result = 1;
else {
    // Переводим в int
    char* endptr; // Указатель на первый непрочитанный символ
    num = strtol(user_input, &endptr, 10);
    // Если есть непрочитанный символ - результат 3
    if (*endptr != '\0') result = 3;
    // Если строка пустая - результат 2
    else if (endptr == user_input) result = 2;
    // Если значение выходит за границы - результат 4
    else if (num <= INT_MIN || INT_MAX <= num) result = 4;
}
if (!result) *buffer = (int)num;
return result;
}

// Безопасный ввод double
int double_input(double* buffer) {
    /*
    buffer: указатель на double, в который нужно записать ввод

    Возвращает:
    0: успешное чтение
    1: выход из ввода по "###"
    2: введена пустая строка
    3: есть непрочитанный символ
    4: значение ввода выходит за границы double
    */
    char user_input[33]; // Буфер для ввода пользователя
    fgets(user_input, 33, stdin); // Ввод пользователя
    // Очищаем вводной поток, если строчка не влезла
    if (user_input[strlen(user_input) - 1] != '\n')
        while (getchar() != '\n');
    else // Удаляем '\n' с конца
        user_input[strlen(user_input) - 1] = '\0';

    int result = 0; // Результат выполнения функции
    double num; // Число пользователя
    // Проверяем на выход из ввода "###" - результат 1
    if (!strcmp(user_input, "###")) result = 1;
    else {
        // Переводим в double
        char* endptr; // Указатель на первый непрочитанный символ
        num = strtod(user_input, &endptr);
        // Если есть непрочитанный символ - результат 3
        if (*endptr != '\0') result = 3;
        // Если строка пустая - результат 2
        else if (endptr == user_input) result = 2;
        // Если значение выходит за границы - результат 4
        else if (num == HUGE_VAL || num == -HUGE_VAL) result = 4;
    }
    if (!result) *buffer = num;
    return result;
}

// Безопасный ввод массива цифр
int input_number_list(int list[], int *len, int start, int end) {
    /*
    list: список для хранения цифр
    len: указатель на переменную, в которую сохраним длину массива
    start: минимальная цифра массива
    end: максимальная цифра массива
    */

```

```

Возвращает:
0: успешное чтение
1: выход из ввода по "###"
2: введена пустая строка
3: некорректный ввод
*/
char user_input[20]; // Буфер для ввода
char* p_input = user_input; // Указатель на буфер

// Если строка пустая - результат 2
if (input(&p_input, 19)) return 2;
// Проверяем на выход из ввода "###" - результат 1
if (!strcmp(user_input, "###")) return 1; // Закончить ввод

// Сохраним все допустимые цифры
char symbols[13]; // Массив всех допустимых цифр
int last_ind = 0; // Индекс последнего элемента массива
for (int i = start; i <= end && last_ind < 10; i++)
    symbols[last_ind++] = '0' + i;
symbols[last_ind++] = ' '; symbols[last_ind++] = ','; symbols[last_ind++] = '\\0';
last_ind = 0; // Используем этот счётчик ещё раз в будущем

// Проверка на лишние символы - результат 3
if (strcspn(user_input, symbols)) return 3;

char* number; // Указатель на цифру для strtok
number = strtok(user_input, " ,");
if ((number == NULL) || (strlen(number) > 1)) return 3;
list[last_ind++] = *number - '0';
while ((number = strtok(NULL, " ,")) != NULL) {
    if (strlen(number) > 1) return 3;
    int cur_number = *number - '0';
    // Проверим, есть ли эта цифра в списке
    int in_list = 0;
    for (int i = 0; i < last_ind; i++)
        if (list[i] == cur_number) in_list = 1;
    if (!in_list) list[last_ind++] = cur_number;
}
*len = last_ind; // Сохраняем длину
return 0;
}

```

Код файла structures.cpp для основных функций для структуры:

```

#define _CRT_SECURE_NO_WARNINGS

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <math.h>
#include "functions.h" // Вспомогательные функции
#include <conio.h>

#include "structure.h"

// Ввод поля fnum
int input_fnum(Flight* pointer) {
    printf("\nВведите номер рейса: ");
    int result; // Результат ввода
    do {
        result = int_input(&(*pointer).fnum);
        if (result > 2) printf("Некорректный ввод. Введите номер рейса: ");
    } while (result > 1);
    return result;
}

// Ввод поля name
int input_name(Flight* pointer) {
    printf("\nВведите тип самолёта: ");
    int result; // Результат ввода
    char user_input[16]; // Буфер для ввода
    char* p_input = user_input; // Указатель на буфер
    do result = input(&p_input);
    while (result); // Ввод, пока не получим непустую строку
    result = !strcmp(user_input, "###");
    if (result == 0) strcpy((*pointer).name, user_input);
    return result;
}

```

```

}

// Ввод поля dest
int input_dest(Flight* pointer) {
    printf("\nВведите пункт назначения: ");
    int result; // Результат ввода
    char user_input[16]; // Буфер для ввода
    char* p_input = user_input; // Указатель на буфер
    do result = input(&p_input);
    while (result); // Ввод, пока не получим непустую строку
    result = !strcmp(user_input, "###");
    if (result == 0) strcpy((*pointer).dest, user_input);
    return result;
}

// Ввод поля days
int input_days(Flight* pointer) {
    int days[7]; // Массив номеров дней
    int last_day = 0; // Индекс последнего элемента массива
    printf("\nВведите дни отправления через запятую: ");
    int result = 0; // Результат ввода
    char user_input[16]; // Буфер для ввода
    char* p_input = user_input; // Указатель на буфер
    do {
        if (result == 3) printf("Некорректный ввод. "
                                "Введите дни отправления через запятую: ");
        result = input_number_list(days, &last_day);
        if (result == 1) return 1; // Закончить ввод
    } while (result);
    // Сортировка пузырьком
    for (int k = 1; k < last_day; k++)
        for (int i = 0; i < last_day - k; i++)
            if (days[i] > days[i + 1]) {
                int buffer = days[i];
                days[i] = days[i + 1];
                days[i + 1] = buffer;
            }
    // Запись в структуру
    for (int i = 0; i < last_day; i++)
        (*pointer).days[i] = days[i];
    // Всё остальное заполняем нулями
    for (int i = last_day; i < 7; i++)
        (*pointer).days[i] = 0;
    return 0;
}

// Ввод поля ***_time
int input_time(Flight* pointer, int field) {
    // Строка на вывод
    const char* info[] = { "Введите время вылета: ",
                            "Введите время прилёта: " };
    int time[2]; // Массив для времени (часы и минуты)
    printf("\n%s", info[field]);
    int result = 0; // Результат ввода
    char user_input[16]; // Буфер для ввода
    char* p_input = user_input; // Указатель на буфер
    do {
        if (result) printf("Некорректный ввод. %s", info[field]);
        do result = input(&p_input);
        while (result); // Ввод, пока не получим непустую строку
        if (!strcmp(user_input, "###")) return 1; // Закончить ввод
        // Проверим на лишние символы
        if (strcspn(user_input, " :0123456789")) { result = 1; continue; }
        char* parts[2]; // Разделим ввод на несколько частей
        parts[0] = strtok(user_input, " :.");
        parts[1] = strtok(NULL, " :.");
        // Строка должна поделиться на две части, а третьей быть NULL
        if ((parts[1] == NULL) || (strtok(NULL, " :.") != NULL)) { result = 1; continue; }
        // Длина первой части должна быть 1 или 2
        switch (strlen(parts[0])) {
            case 1: { time[0] = parts[0][0] - '0'; break; }
            case 2: { time[0] = (parts[0][0] - '0') * 10 + parts[0][1] - '0'; break; }
            default: { result = 1; break; }
        }
        // Длина второй всегда 2
        if (strlen(parts[1]) != 2) { result = 1; continue; }
        time[1] = (parts[1][0] - '0') * 10 + parts[1][1] - '0';
        // Проверим на корректные значения
    }
}

```

```

        if (time[0] > 23 || time[1] > 59) result = 1;
    } while (result);
    switch (field) {
    case 0: { (*pointer).dep_time = time[0] * 60 + time[1]; break; }
    case 1: { (*pointer).arr_time = time[0] * 60 + time[1]; break; }
    }
    return 0;
}

// Ввод поля price
int input_price(Flight* pointer) {
    printf("\nВведите цену билета: ");
    int result; // Результат ввода
    do {
        result = double_input(&(*pointer).price);
        if (result > 2) printf("Некорректный ввод. Введите цену билета: ");
    } while (result > 1 || (*pointer).price < 0);
    return result;
}

// Ввод полной структуры
int input_structure(Flight* pointer) {
    printf("\nВведите запись по столбцам. "
        "Чтобы закончить ввод, введите \"###\");
    int result = 0;
    // Ввод всех полей
    if (input_fnum(pointer) || input_name(pointer) ||
        input_dest(pointer) || input_days(pointer) ||
        input_time(pointer, 0) || input_time(pointer, 1) ||
        input_price(pointer))
    {
        printf("\nВвод завершён, запись не была добавлена.");
        result = 1;
    }
    else printf("\nВвод завершён, запись добавлена в таблицу.");
    return result;
}

// Вывод одной структуры
void print_structure(Flight flight) {
    // Получим длины каждого поля
    int len_fnum = 1;
    if (flight.fnum) len_fnum = log10(flight.fnum) + 1;
    int len_name = strlen(flight.name);
    int len_dest = strlen(flight.dest);
    int len_dep_time = (flight.dep_time / 60 >= 10) + 1;
    int len_arr_time = (flight.arr_time / 60 >= 10) + 1;
    int len_price = 4;
    if (flight.price > 0) len_price = log10(flight.price) + 1 + 3;
    int len_days = 0;
    for (int i = 0; flight.days[i] && i < 7; i++)
        len_days += 2;
    len_days -= 1;
    // Теперь выведем каждое поле
    // Номер рейса
    printf("|"); for (int i = 0; i < 7 - len_fnum / 2; i++) printf(" ");
    printf("%d", flight.fnum);
    for (int i = 0; i < (7 - (len_fnum - 1) / 2); i++) printf(" ");
    // Тип самолёта
    printf("|"); for (int i = 0; i < 7 - len_name / 2; i++) printf(" ");
    printf("%s", flight.name);
    for (int i = 0; i < 7 - (len_name - 1) / 2; i++) printf(" ");
    // Пункт назначения
    printf("|"); for (int i = 0; i < 7 - len_dest / 2; i++) printf(" ");
    printf("%s", flight.dest);
    for (int i = 0; i < 7 - (len_dest - 1) / 2; i++) printf(" ");
    // Дни отправления
    printf("|%*c", (15 - len_days) / 2, ' ', flight.days[0]);
    for (int i = 1; flight.days[i] && i < 7; i++) printf(",%d", flight.days[i]);
    printf("%*c", (16 - len_days) / 2, ' ');
    // Время вылета
    printf("|%*c", 7 - len_dep_time, ' ');
    printf("%d:%.2d", flight.dep_time / 60, flight.dep_time % 60);
    printf("%*c", 7 - 2, ' ');
    // Время прилёта
    printf("|%*c", 7 - len_arr_time, ' ');
    printf("%d:%.2d", flight.arr_time / 60, flight.arr_time % 60);
    printf("%*c", 7 - 2, ' ');
    // Цена билета

```

```

    if (len_price <= 13) {
        printf("|%*c", 7 - len_price / 2, ' ');
        printf("%.2lf", flight.price);
        printf("%*c", 7 - (len_price - 1) / 2, ' ');
    }
    else printf("|%9.9e", flight.price);
    printf("\n");
}

// Печать границы таблицы
void print_edge() {
    printf("+");
    for (int i = 1; i < 8 * (16); i++)
        i % 16 ? printf("-") : printf("+");
    printf("+\n");
}

// Печать отступа таблицы
void print_space() {
    printf("|");
    for (int i = 1; i < 8 * (16); i++)
        i % 16 ? printf(" ") : printf("|");
    printf("\n");
}

// Печать шапки таблицы
void print_head() {
    printf("Список авиарейсов:\n");
    // Шапка таблицы
    print_edge();
    // Первая строка
    printf("|    Номер    |    Номер    |    Тип    |    Пункт    "
           "|    Дни     |    Время    |    Время    |    Цена     "
           "\n");
    // Вторая строка
    printf("|    в таблице |    рейса    |    самолёта |    назначения "
           "|    отправления |    вылета    |    прилёта    |    билета     "
           "\n");
    print_edge();
}

// Соответствует ли запись заданным фильтрам
int compare_flight(Flight flight, Flight_filter filter) {
    int is_good = 1; // Запись изначально подходит
    // Сверяем номера рейсов
    if (filter.apply[0]) {
        char flight_fnum[16], filter_fnum[16];
        _itoa(flight.fnum, flight_fnum, 10);
        _itoa(filter.fnum, filter_fnum, 10);
        if (strstr(flight_fnum, filter_fnum) == NULL) is_good = 0;
    }
    // Сверяем типы самолётов
    if (is_good && filter.apply[1]) {
        char flight_name[16], filter_name[16];
        _strlwr(strcpy(flight_name, flight.name));
        _strlwr(strcpy(filter_name, filter.name));
        if (strstr(flight_name, filter_name) == NULL) is_good = 0;
    }
    // Сверяем пункты назначения
    if (is_good && filter.apply[2]) {
        char flight_dest[16], filter_dest[16];
        _strlwr(strcpy(flight_dest, flight.dest));
        _strlwr(strcpy(filter_dest, filter.dest));
        if (strstr(flight_dest, filter_dest) == NULL) is_good = 0;
    }
    // Сверяем дни отправления
    if (is_good && filter.apply[3]) {
        int common = 0;
        for (int i = 0; i < 7 && filter.days[i]; i++)
            for (int j = 0; j < 7 && flight.days[j]; j++)
                if (filter.days[i] == flight.days[j]) common += 1;
        if (!common) is_good = 0;
    }
    // Сверяем время вылета
    if (is_good && filter.apply[4]) {
        if (filter.apply[4] == 1) // flight > filter
            if (flight.dep_time <= filter.dep_time) is_good = 0;
        if (filter.apply[4] == 2) // flight < filter
            if (flight.dep_time >= filter.dep_time) is_good = 0;
    }
}

```

```

        if (filter.apply[4] == 3) // flight = filter
            if (flight.dep_time != filter.dep_time) is_good = 0;
        if (filter.apply[4] == 4) // flight >= filter
            if (flight.dep_time < filter.dep_time) is_good = 0;
        if (filter.apply[4] == 5) // flight <= filter
            if (flight.dep_time > filter.dep_time) is_good = 0;
    }
    // Сверяем время прилёта
    if (is_good && filter.apply[5]) {
        if (filter.apply[5] == 1) // flight > filter
            if (flight.arr_time <= filter.arr_time) is_good = 0;
        if (filter.apply[5] == 2) // flight < filter
            if (flight.arr_time >= filter.arr_time) is_good = 0;
        if (filter.apply[5] == 3) // flight = filter
            if (flight.arr_time != filter.arr_time) is_good = 0;
        if (filter.apply[5] == 4) // flight >= filter
            if (flight.arr_time < filter.arr_time) is_good = 0;
        if (filter.apply[5] == 5) // flight <= filter
            if (flight.arr_time > filter.arr_time) is_good = 0;
    }
    // Сверяем цену билета
    if (is_good && filter.apply[6]) {
        if (filter.apply[6] == 1) // flight > filter
            if (flight.price <= filter.price) is_good = 0;
        if (filter.apply[6] == 2) // flight < filter
            if (flight.price >= filter.price) is_good = 0;
        if (filter.apply[6] == 3) // flight = filter
            if (flight.price != filter.price) is_good = 0;
        if (filter.apply[6] == 4) // flight >= filter
            if (flight.price < filter.price) is_good = 0;
        if (filter.apply[6] == 5) // flight <= filter
            if (flight.price > filter.price) is_good = 0;
    }
    return is_good;
}

// Вывод массива структур
int print_structures(Flight* flights, int len, Flight_filter filter) {
    if (len == 0) { printf("Таблица на данный момент пустая."); return -1; }
    int counter = 0;
    for (int line = 0; line < len; line++) {
        // Выведем номер в таблице
        if (!compare_flight(flights[line], filter)) continue;
        if (!counter) print_head(); // Печатаем верх таблицы, если ещё не печатали
        int len_line = log10(line + 1) + 1;
        printf("%*c", 7 - len_line / 2, ' ');
        printf("%d", line + 1);
        printf("%*c", 7 - (len_line - 1) / 2, ' ');
        print_structure(flights[line]);
        print_edge();
        counter += 1;
    }
    if (!counter) printf("\nНет записей, соответствующих заданным фильтрам.");
    return counter;
}

// Изменение одной структуры
int edit_structure(Flight* pointer) {
    printf("\nВведите номера столбцов, которые вы хотите изменить, через запятую:\n");
    printf("1 - Номер рейса\n");
    printf("2 - Тип самолёта\n");
    printf("3 - Пункт назначения\n");
    printf("4 - Дни отправления\n");
    printf("5 - Время вылета\n");
    printf("6 - Время прилёта\n");
    printf("7 - Цена билета\n");
    int cols[7]; // Массив номеров столбцов
    int last_ind = 0; // Индекс последнего элемента массива
    int cols_res = 0; // Результат ввода массива столбцов
    char user_input[16]; // Буфер для ввода
    char* p_input = user_input; // Указатель на буфер
    do {
        if (cols_res == 3) printf("Некорректный ввод. "
            "\nВведите номера столбцов, которые вы хотите изменить, через запятую: ");
        cols_res = input_number_list(cols, &last_ind);
    } while (cols_res > 1);
    if (cols_res == 1) return 2; // Если нужно остановить ввод
    // Сортировка пузырьком

```



```

for (int k = 1; k < last_ind; k++)
    for (int i = 0; i < last_ind - k; i++)
        if (cols[i] > cols[i + 1]) {
            int buffer = cols[i];
            cols[i] = cols[i + 1];
            cols[i + 1] = buffer;
        }
// Меняем столбцы
int change_res = 0; // Результат ввода новых значений
for (int i = 0; i < last_ind && !change_res; i++)
    if (cols[i] == 1) change_res = input_fnum(pointer);
    else if (cols[i] == 2) change_res = input_name(pointer);
    else if (cols[i] == 3) change_res = input_dest(pointer);
    else if (cols[i] == 4) change_res = input_days(pointer);
    else if (cols[i] == 5) change_res = input_time(pointer, 0);
    else if (cols[i] == 6) change_res = input_time(pointer, 1);
    else if (cols[i] == 7) change_res = input_price(pointer);
return change_res; // Если ввод полный, то 0 - иначе 1
}

```

Код основного файла:

```

#define _CRT_SECURE_NO_WARNINGS

#include <stdio.h> // Стандартный вход/выход
#include <locale.h> // Вывод на русском языке
#include <conio.h> // Задержка экрана

#include <string.h> // Работа со строками
#include <math.h> // Математические функции

#include <stdlib.h> // Создание динамических массивов, использование рандома и преобразования строк
#include <windows.h> // Очистка консоли и установка кодировки

#include "structure.h" // Работа со структурой
#include "functions.h" // Вспомогательные функции

#define SETIO SetConsoleCP(1251); SetConsoleOutputCP(1251); // Ввод и вывод на русском
#define START_MEM 20 // Стартовое количество мест в таблице (сколько памяти выделяется)

// Увеличение места в таблице
Flight* extend_table(Flight* table, int old_len, int new_len) {
    Flight* new_table = (Flight*)calloc(new_len, sizeof(Flight));
    for (int i = 0; i < old_len; i++) new_table[i] = table[i];
    free(table);
    return new_table;
}

// Выбор первичного действия
int get_first_action(Flight* table, int len, Flight_filter filter) {
    // Ввод пока не получим число от 1 до 4
    int action; // Выбор пользователя
    do {
        system("cls");
        printf("Авиарейсы MARK company:\n");
        int counter = 0; // Посчитаем количество подходящих записей
        for (int line = 0; line < len; line++)
            if (compare_flight(table[line], filter)) counter++;
        if (counter) print_structures(table, len, filter);
        printf("\nВыберите действие:\n");
        printf("1 - Вывод таблицы\n");
        printf("2 - Ввод и корректировка списка\n");
        printf("3 - Обработка запроса по списку\n");
        printf("4 - Выход из программы\n");
    } while (int_input(&action) || (action < 1 || 4 < action));
    return action;
}

// Выбор действия по вводу и корректировке списка
int get_edit_action(Flight* table, int len, Flight_filter filter) {
    // Ввод пока не получим число от 1 до 4
    int action; // Выбор пользователя
    do {
        system("cls");
        printf("Авиарейсы MARK company:\n");
        int counter = 0; // Посчитаем количество подходящих записей
        for (int line = 0; line < len; line++)
            if (compare_flight(table[line], filter)) counter++;
    }

```

```

        if (counter) print_structures(table, len, filter);
        printf("\nВыберите действие по вводу и корректировке:\n");
        printf("1 - Ввод таблицы\n");
        printf("2 - Изменение записи по номеру\n");
        printf("3 - Удаление записей по номерам\n");
        printf("4 - Вернуться в главное меню\n");
    } while (int_input(&action) || (action < 1 || 4 < action));
    return action;
}

int main() {
    system("color F0");
    system("mode con cols=130 lines=40");
    SETIO;

    int max_len = START_MEM; // Максимальное количество записей в таблице
    Flight* table = (Flight*)calloc(max_len, sizeof(Flight));
    int len = 0; // Текущее количество записей в таблице

    Flight_filter filter; // Создаём фильтр
    // Обнуляем фильтр
    filter.apply[0] = filter.apply[1] = filter.apply[2] = filter.apply[3] = 0;
    filter.apply[4] = filter.apply[5] = filter.apply[6] = 0;

    int run = 1; // Программа работает
    while (run) {
        int first_action = get_first_action(table, len, filter);
        // 1 - Вывод таблицы
        if (first_action == 1) {
            system("cls");
            print_structures(table, len, filter);
            printf("\nНажмите любую кнопку, чтобы продолжить");
            _getch();
        }
        // 2 - Ввод и корректировка списка
        else if (first_action == 2) {
            int edit_action = get_edit_action(table, len, filter);
            // 1 - Ввод таблицы
            if (edit_action == 1) {
                int n; // Сколько строчек нужно добавить
                int res; // Результат ввода
                do printf("\nВведите количество строчек, которые хотите добавить (1-99): ");
                while ((res = int_input(&n)) > 1 || (n < 1 || 99 < n));
                if (res == 1) printf("\nВвод строк отменён.");
                else {
                    // Проверяем, хватает ли места
                    if (len + n > max_len) {
                        max_len = (max_len + n) + 5 - ((max_len + n) % 5); // Округлим до
                    }
                    table = extend_table(table, len, max_len);
                }
                int result = 0; // Результат ввода последней строчки
                int old_len = len; // Сохраняем длину до ввода
                for (int i = 0; !result && i < n; i++) {
                    Flight new_line;
                    Flight* p_nl = &new_line;
                    result = input_structure(p_nl);
                    if (!result) table[len++] = new_line;
                }
                printf("\nВвод строк закончен. Добавлено строк: %d", len - old_len);
            }
            printf("\nНажмите любую кнопку, чтобы продолжить");
            _getch();
        }
        // 2 - Изменение записи по номеру
        else if (edit_action == 2) {
            do {
                system("cls");
                print_structures(table, len, filter);
                printf("\n");
                if (len == 0) break;
                int n; // Какую запись нужно поменять
                int n_res; // Результат ввода n
                do {
                    printf("\nВведите номер записи, которую хотите поменять (1-%d): ",
len);

                    n_res = int_input(&n);
                } while (n_res > 1 || (n < 1 || len < n));
                if (n_res == 1) { // Если нужно остановить ввод

```

значений

запятую: ", len);

```
        printf("\nИзменение записи отменено.");
        break;
    }
    Flight* pointer = table + n - 1; // Указатель на нужную запись
    int change_res = edit_structure(pointer); // Результат ввода новых

    if (change_res == 0) printf("\nЗапись успешно изменена.");
    else if (change_res == 1) printf("\nЗапись изменена лишь частично.");
    else if (change_res == 2) printf("\nИзменение записи отменено.");
} while (0); // Костыль для удобного выхода в нужный момент
printf("\nНажмите любую кнопку, чтобы продолжить");
_getch();
}
// 3 - Удаление записей по номерам
else if (edit_action == 3) {
    do {
        // Просим пользователя ввести строчку
        system("cls");
        print_structures(table, len, filter);
        printf("\n");
        if (len == 0) break;
        printf("\nВведите номера записей (1-%d), которые хотите удалить, через

        int buffer_len = len * ((int)log10(len) + 3) + 4;
        char* user_input = (char*)calloc(buffer_len, sizeof(char));
        if (user_input == NULL) {
            printf("\nОшибка выделения памяти.");
            break;
        }
        fgets(user_input, buffer_len, stdin);

        // Очищаем вводной поток, если строка не влезла
        if (user_input[strlen(user_input) - 1] != '\n') {
            while (getchar() != '\n');
            printf("\nВведена слишком длинная строка.\n");
            printf("Попробуйте убрать пробелы и повторяющиеся числа.\n");
            break;
        }
        else // Удаляем '\n' с конца
            user_input[strlen(user_input) - 1] = '\0';

        // Проверяем на выход из ввода "###" - результат 1
        if (!strcmp(user_input, "###")) {
            printf("\nУдаление записей отменено.");
            free(user_input);
            break;
        }
        // Проверка на лишние символы - результат 3
        if (strcspn(user_input, " ,0123456789")) {
            printf("\nНекорректный ввод.");
            free(user_input);
            break;
        }

        // Объявляем нужные переменные
        int* to_del = (int*)calloc(len, sizeof(int)); // Массив для номеров
        if (to_del == NULL) {
            printf("\nОшибка выделения памяти.");
            free(user_input);
            break;
        }
        int last_ind = 0; // Индекс последнего элемента
        char* number; // Строка-число для strtok
        char* endptr; // Указатель на символ для strtol

        // Начинаем разбирать строчку на номера записей
        number = strtok(user_input, " ,");
        if (number == NULL) {
            printf("\nНекорректный ввод.");
            free(user_input);
            free(to_del);
            break;
        }
        do {
            long cur_num_long = strtol(number, &endptr, 10);
            // Проверяем на корректность числа
            if (cur_num_long < 1 || len < cur_num_long) continue;
            int cur_num = (int)cur_num_long;
            int in_list = 0; // Есть ли число в массиве
```

```

        for (int i = 0; i < last_ind; i++)
            if (to_del[i] == cur_num) in_list = 1;
        if (!in_list) to_del[last_ind++] = cur_num;
    } while ((number = strtok(NULL, " ,")) != NULL);
    free(user_input);

    // Просим у пользователя подтверждение операции
    printf("\nВы уверены, что хотите удалить следующие записи:\n%d",
to_del[0]);

    for (int i = 1; i < last_ind; i++) printf(", %d", to_del[i]);
    printf("\n\n0 - Отмена\n");
    printf("1 - Удалить\n");
    char confirm[4]; // Буфер для ввода
    char* p_conf = confirm; // Указатель на буфер ввода
    if (input(&p_conf, 3) || strcmp(confirm, "1")) {
        free(to_del);
        printf("\nУдаление отменено.");
        break;
    }

    // Сортируем массив по убыванию
    for (int k = 1; k < last_ind; k++)
        for (int i = 0; i < last_ind - k; i++)
            if (to_del[i] < to_del[i + 1]) {
                int buffer = to_del[i];
                to_del[i] = to_del[i + 1];
                to_del[i + 1] = buffer;
            }

    // Удаляем строочки
    for (int i = 0; i < last_ind; i++) {
        for (int j = to_del[i] - 1; j < len - 1; j++)
            table[j] = table[j + 1];
        len -= 1;
    }
    printf("\nУдаление завершено.");
} while (0);
printf("\nНажмите любую кнопку, чтобы продолжить");
_getch();
}
// 4 - Вернуться в главное меню
else if (edit_action == 4);
}
// 3 - Обработка запроса по списку
else if (first_action == 3) {
    do {
        system("cls");
        if (print_structures(table, len, filter) == -1) break;
        printf("\n\nТекущие фильтры:");

        printf("\n1. Номер рейса - ");
        if (filter.apply[0]) printf("%d", filter.fnum);
        else printf("не используется");

        printf("\n2. Тип самолёта - ");
        if (filter.apply[1]) printf("%s", filter.name);
        else printf("не используется");

        printf("\n3. Пункт назначения - ");
        if (filter.apply[2]) printf("%s", filter.dest);
        else printf("не используется");

        printf("\n4. Дни отправления - ");
        if (filter.apply[3]) {
            printf("%d", filter.days[0]);
            for (int i = 1; i < 7 && filter.days[i]; i++)
                printf(", %d", filter.days[i]);
        }
        else printf("не используется");

        printf("\n5. Время вылета - ");
        if (filter.apply[4]) {
            if (filter.apply[4] == 1) printf(">");
            else if (filter.apply[4] == 2) printf("<");
            else if (filter.apply[4] == 3) printf("=");
            else if (filter.apply[4] == 4) printf(">=");
            else if (filter.apply[4] == 5) printf("<=");
            printf("%d:%.2d", filter.dep_time / 60, filter.dep_time % 60);
        }
    } while (1);
}

```

```

}
else printf("не используется");

printf("\n6. Время прилёта - ");
if (filter.apply[5]) {
    if (filter.apply[5] == 1) printf(">");
    else if (filter.apply[5] == 2) printf("<");
    else if (filter.apply[5] == 3) printf("=");
    else if (filter.apply[5] == 4) printf(">=");
    else if (filter.apply[5] == 5) printf("<=");
    printf("%d:%.2d", filter.arr_time / 60, filter.arr_time % 60);
}
else printf("не используется");

printf("\n7. Цена билета - ");
if (filter.apply[6]) {
    if (filter.apply[6] == 1) printf(">");
    else if (filter.apply[6] == 2) printf("<");
    else if (filter.apply[6] == 3) printf("=");
    else if (filter.apply[6] == 4) printf(">=");
    else if (filter.apply[6] == 5) printf("<=");
    printf("%.2lf", filter.price);
}
else printf("не используется");

printf("\n\nВведите 0, чтобы вернуться в главное меню.\n");
printf("Введите номер столбца, чтобы настроить ему фильтр: ");
int n, res; // Выбранный столбец и результат ввода
res = int_input(&n);
if (res || n == 0) break;

printf("\n\nВ любой момент вы можете ввести ### для отмены.\n");
if (filter.apply[n - 1]) {
    printf("\nВведите 0, чтобы удалить фильтр.\n");
    printf("Введите 1, чтобы изменить текущий\n");
    int choice;
    do res = int_input(&choice);
    while (res > 1 || (choice != 0 && choice != 1));
    if (res == 1) break;
    if (choice == 0) {
        filter.apply[n - 1] = 0;
        printf("\nФильтр удалён.");
        break;
    }
}

int fil_result; // Результат ввода фильтра
Flight holder; // Для удобного ввода
if (n == 1) {
    fil_result = input_fnum(&holder);
    filter.fnum = holder.fnum;
    filter.apply[0] = !fil_result;
}
if (n == 2) {
    fil_result = input_name(&holder);
    strcpy(filter.name, holder.name);
    filter.apply[1] = !fil_result;
}
if (n == 3) {
    fil_result = input_dest(&holder);
    strcpy(filter.dest, holder.dest);
    filter.apply[2] = !fil_result;
}
if (n == 4) {
    fil_result = input_days(&holder);
    for (int i = 0; i < 7; i++)
        filter.days[i] = holder.days[i];
    filter.apply[3] = !fil_result;
}
if (n == 5) {
    fil_result = input_time(&holder, 0);
    filter.dep_time = holder.dep_time;
    if (!fil_result) {
        printf("\nВведите оператор сравнения:");
        printf("\n1. \">>" (Позже %d:%.2d)",
            filter.dep_time / 60, filter.dep_time % 60);
        printf("\n2. \"><" (Раньше %d:%.2d)",
            filter.dep_time / 60, filter.dep_time % 60);
    }
}

```

```

        printf("\n3. \"=\" (Точно в %d:%.2d)",
            filter.dep_time / 60, filter.dep_time % 60);
        printf("\n4. \">=\" (Позже или в %d:%.2d)",
            filter.dep_time / 60, filter.dep_time % 60);
        printf("\n5. \"<=\" (Раньше или в %d:%.2d)\n",
            filter.dep_time / 60, filter.dep_time % 60);

        int comp; // Выбранный оператор
        int comp_res; // Результат ввода оператора
        do comp_res = int_input(&comp);
        while (comp_res > 1 || (comp < 1 || 5 < comp));
        if (comp_res) filter.apply[4] = 0;
        else filter.apply[4] = comp;
    }
    else filter.apply[4] = 0;
}
if (n == 6) {
    fil_result = input_time(&holder, 1);
    filter.arr_time = holder.arr_time;
    if (!fil_result) {
        printf("\nВведите оператор сравнения:");
        printf("\n1. \">\" (Позже %d:%.2d)",
            filter.arr_time / 60, filter.arr_time % 60);
        printf("\n2. \"<\" (Раньше %d:%.2d)",
            filter.arr_time / 60, filter.arr_time % 60);
        printf("\n3. \"=\" (Точно в %d:%.2d)",
            filter.arr_time / 60, filter.arr_time % 60);
        printf("\n4. \">=\" (Позже или в %d:%.2d)",
            filter.arr_time / 60, filter.arr_time % 60);
        printf("\n5. \"<=\" (Раньше или в %d:%.2d)\n",
            filter.arr_time / 60, filter.arr_time % 60);

        int comp; // Выбранный оператор
        int comp_res; // Результат ввода оператора
        do comp_res = int_input(&comp);
        while (comp_res > 1 || (comp < 1 || 5 < comp));
        if (comp_res) filter.apply[5] = 0;
        else filter.apply[5] = comp;
    }
    else filter.apply[5] = 0;
}
if (n == 7) {
    fil_result = input_price(&holder);
    filter.price = holder.price;
    if (!fil_result) {
        printf("\nВведите оператор сравнения:");
        printf("\n1. \">\" (Дороже %.2lf)", filter.price);
        printf("\n2. \"<\" (Дешевле %.2lf)", filter.price);
        printf("\n3. \"=\" (Ровно %.2lf)", filter.price);
        printf("\n4. \">=\" (Дороже или ровно %.2lf)", filter.price);
        printf("\n5. \"<=\" (Дешевле или ровно %.2lf)\n", filter.price);

        int comp; // Выбранный оператор
        int comp_res; // Результат ввода оператора
        do comp_res = int_input(&comp);
        while (comp_res > 1 || (comp < 1 || 5 < comp));
        if (comp_res) filter.apply[6] = 0;
        else filter.apply[6] = comp;
    }
    else filter.apply[6] = 0;
}
printf("\nФильтр успешно добавлен.");
} while (0);
printf("\nНажмите любую кнопку, чтобы продолжить");
_getch();
}
// 4 - Выход из программы
else if (first_action == 4) run = 0;
}
free(table);
}

```

Тесты программы

Начальное меню:

Авиарейсы MARK company:

Выберите действие:

- 1 - Вывод таблицы
- 2 - Ввод и корректировка списка
- 3 - Обработка запроса по списку
- 4 - Выход из программы

Введём данные:

Из примера:

Авиарейсы MARK company:

Выберите действие по вводу и корректировке:

- 1 - Ввод таблицы
 - 2 - Изменение записи по номеру
 - 3 - Удаление записей по номерам
 - 4 - Вернуться в главное меню
- 1

Введите количество строчек, которые хотите добавить (1-99): 1

Введите запись по столбцам. Чтобы закончить ввод, введите "###"

Введите номер рейса: 121

Введите тип самолёта: Ту-154

Введите пункт назначения: Москва

Введите дни отправления через запятую: 1,3,5

Введите время вылета: 10.00

Введите время прилёта: 14.00

Введите цену билета: 5000.20

Ввод завершён, запись добавлена в таблицу.

Ввод строк закончен. Добавлено строк: 1

Нажмите любую кнопку, чтобы продолжить

Авиарейсы MARK company:

Список авиарейсов:

Номер в таблице	Номер рейса	Тип самолёта	Пункт назначения	Дни отправления	Время вылета	Время прилёта	Цена билета
1	121	Ту-154	Москва	1,3,5	10:00	14:00	5000.20

Максимально маленький набор:

Пункты 1 и 3 при пустой таблице:

Таблица на данный момент пустая.

Нажмите любую кнопку, чтобы продолжить

Введите количество строчек, которые хотите добавить (1-99): 1

Введите запись по столбцам. Чтобы закончить ввод, введите "###"

Введите номер рейса: Нечисло

Некорректный ввод. Введите номер рейса:

99999999999999999999

Некорректный ввод. Введите номер рейса: 99

Введите тип самолёта:

Як-99

Введите пункт назначения:

Москва

Введите дни отправления через запятую: Нечисла

Некорректный ввод. Введите дни отправления через запятую:

9

Некорректный ввод. Введите дни отправления через запятую: 5,6

Введите время вылета: Невремя

Некорректный ввод. Введите время вылета:

12

Некорректный ввод. Введите время вылета: 24.00

Некорректный ввод. Введите время вылета: 23.60

Некорректный ввод. Введите время вылета: 12.1

Некорректный ввод. Введите время вылета: 9.00

Введите время прилёта: 9.00

Введите цену билета: Нечисло

Некорректный ввод. Введите цену билета:

500

Ввод завершён, запись добавлена в таблицу.

Ввод строк закончен. Добавлено строк: 1

Нажмите любую кнопку, чтобы продолжить

Авиарейсы MARK company:

Список авиарейсов:

Номер в таблице	Номер рейса	Тип самолёта	Пункт назначения	Дни отправления	Время вылета	Время прилёта	Цена билета
1	121	Ту-154	Москва	1,3,5	10:00	14:00	5000.20
2	1	Я	Н	1	1:00	1:00	0.00
3	99999999	ОЧЕНЬБОЛЬШОЕНАЗ	ОЧЕНЬДЛИННОЕНАЗ	1,2,3,4,5,6,7	23:59	23:59	1.000000000e+28
4	99	Як-99	Москва	5,6	9:00	9:00	500.00

Вывод непустой таблицы:

Список авиарейсов:

Номер в таблице	Номер рейса	Тип самолёта	Пункт назначения	Дни отправления	Время вылета	Время прилёта	Цена билета
1	121	Ту-154	Москва	1,3,5	10:00	14:00	5000.20
2	1	Я	Н	1	1:00	1:00	0.00
3	99999999	ОЧЕНЬБОЛЬШОЕНАЗ	ОЧЕНЬДЛИННОЕНАЗ	1,2,3,4,5,6,7	23:59	23:59	1.000000000e+28
4	99	Як-99	Москва	5,6	9:00	9:00	500.00

Нажмите любую кнопку, чтобы продолжить

Доступен выход в главное меню в любой момент:

Введите количество строчек, которые хотите добавить (1-99): 1

Введите запись по столбцам. Чтобы закончить ввод, введите "###"

Введите номер рейса: 99

Введите тип самолёта: ###

Ввод завершён, запись не была добавлена.

Ввод строк закончен. Добавлено строк: 0

Нажмите любую кнопку, чтобы продолжить

Редактирование списка:

Введите номер записи, которую хотите поменять (1-4): 4

Введите номера столбцов, которые вы хотите изменить, через запятую:

1 - Номер рейса

2 - Тип самолёта

3 - Пункт назначения

4 - Дни отправления

5 - Время вылета

6 - Время прилёта

7 - Цена билета

2

Введите тип самолёта: Spitfire

Запись успешно изменена.

Нажмите любую кнопку, чтобы продолжить

Список авиаарейсов:

Номер в таблице	Номер рейса	Тип самолёта	Пункт назначения	Дни отправления	Время вылета	Время прилёта	Цена билета
1	121	Ty-154	Москва	1,3,5	10:00	14:00	5000.20
2	1	я	н	1	1:00	1:00	0.00
3	99999999	ОЧЕНЬБОЛЬШОЕНАЗ	ОЧЕНЬДЛИННОЕНАЗ	1,2,3,4,5,6,7	23:59	23:59	1.000000000e+28
4	99	Spitfire	Москва	5,6	9:00	9:00	500.00

Удаление записей:

Введите номера записей (1-4), которые хотите удалить, через запятую: 2, 3

Вы уверены, что хотите удалить следующие записи:

2, 3

0 - Отмена

1 - Удалить

1

Удаление завершено.

Нажмите любую кнопку, чтобы продолжить

Авиарейсы MARK company:

Список авиаарейсов:

Номер в таблице	Номер рейса	Тип самолёта	Пункт назначения	Дни отправления	Время вылета	Время прилёта	Цена билета
1	121	Ty-154	Москва	1,3,5	10:00	14:00	5000.20
2	99	Spitfire	Москва	5,6	9:00	9:00	500.00

Меню фильтров:

Текущие фильтры:

1. Номер рейса - не используется
2. Тип самолёта - не используется
3. Пункт назначения - не используется
4. Дни отправления - не используется
5. Время вылета - не используется
6. Время прилёта - не используется
7. Цена билета - не используется

Введите 0, чтобы вернуться в главное меню.

Введите номер столбца, чтобы настроить ему фильтр:

Добавим фильтр на цену:

Введите 0, чтобы вернуться в главное меню.

Введите номер столбца, чтобы настроить ему фильтр: 7

В любой момент вы можете ввести ### для отмены.

Введите цену билета: 1000

Введите оператор сравнения:

1. ">" (Дороже 1000.00)
 2. "<" (Дешевле 1000.00)
 3. "=" (Ровно 1000.00)
 4. ">=" (Дороже или ровно 1000.00)
 5. "<=" (Дешевле или ровно 1000.00)
- 2

Фильтр успешно добавлен.

Нажмите любую кнопку, чтобы продолжить.

Список авиарейсов:

Номер в таблице	Номер рейса	Тип самолёта	Пункт назначения	Дни отправления	Время вылета	Время прилёта	Цена билета
2	99	Spitfire	Москва	5,6	9:00	9:00	500.00

Текущие фильтры:

1. Номер рейса - не используется
2. Тип самолёта - не используется
3. Пункт назначения - не используется
4. Дни отправления - не используется
5. Время вылета - не используется
6. Время прилёта - не используется
7. Цена билета - <1000.00

Введите 0, чтобы вернуться в главное меню.

Введите номер столбца, чтобы настроить ему фильтр:

Удалим фильтр на цену:

Введите 0, чтобы вернуться в главное меню.

Введите номер столбца, чтобы настроить ему фильтр: 7

В любой момент вы можете ввести ### для отмены.

Введите 0, чтобы удалить фильтр.

Введите 1, чтобы изменить текущий

0

Фильтр удалён.

Нажмите любую кнопку, чтобы продолжить.

Список авиарейсов:							
Номер в таблице	Номер рейса	Тип самолёта	Пункт назначения	Дни отправления	Время вылета	Время прилёта	Цена билета
1	121	Ty-154	Москва	1,3,5	10:00	14:00	5000.20
2	99	Spitfire	Москва	5,6	9:00	9:00	500.00

Поставим фильтры, под которые не подходит ни одна запись:

Нет записей, соответствующих заданным фильтрам.

Нажмите любую кнопку, чтобы продолжить.

Ввод фильтров использует те же функции, что и ввод самих записей, поэтому бессмысленно добавлять тесты и сюда.

4 - Выход из программы Мгновенно завершает работу программы